

RANDOM PASSWORD GENERATOR PROJECT

REPORT

Project Title

Random Password Generator

Objective

The objective of this project is to develop a Python-based application that generates strong and random passwords according to user-defined criteria. The program helps users create secure passwords by allowing them to select password length and character types such as uppercase letters, lowercase letters, numbers, and special symbols.

Introduction

Passwords play a crucial role in protecting personal and professional data. Weak passwords can be easily guessed or cracked, leading to security risks. This project provides an automated solution for generating strong and secure passwords. The application ensures that passwords meet user requirements and promotes better cybersecurity practices.

Problem Statement

Many users create simple passwords that are vulnerable to attacks. Creating complex passwords manually can be difficult and time-consuming. Therefore, a tool is needed to generate random and secure passwords efficiently.

Technologies Used

- Python
- random module
- string module

Features

- User-defined password length
- Minimum password length validation (8 characters)
- Selection of character types:
 - Uppercase letters
 - Lowercase letters
 - Numbers
 - Symbols
- Random password generation
- Input validation
- Option to generate multiple passwords without restarting the application

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 2 GB RAM

Software Requirements

- Python 3.x
- Any code editor (PyCharm, VS Code, IDLE)

Methodology

1. Take password length as input from the user.
2. Validate that the length is at least 8 characters.
3. Ask the user which character types should be included.
4. Ensure at least two character categories are selected.
5. Create a character pool based on selected options.
6. Generate a random password using the random module.
7. Display the generated password.
8. Allow the user to generate another password if desired.

Algorithm

1. Start the program.
2. Input password length.
3. Validate password length.
4. Select character categories.
5. Validate category selection.
6. Create character set.
7. Generate random password.
8. Display password.
9. Ask whether to generate another password.
10. Stop or repeat.

Output

The program successfully generates secure passwords according to the user's preferences and displays them on the terminal.

Sample Output

Random Password Generator

Enter password length: 12

Include Uppercase Letters? y

Include Lowercase Letters? y

Include Numbers? y

Include Symbols? y

Generated Password: A7@kP9!mQ2#x

Advantages

- Easy to use
- Saves time
- Generates strong passwords
- Improves security
- Supports customization

Limitations

- Beginner version works only in the command line.
- Passwords are not stored for future use.

Future Enhancements

- Graphical User Interface (GUI)
- Password strength indicator
- Copy-to-clipboard functionality
- Password history
- Use of secrets module for cryptographically secure passwords
- Export generated passwords securely

Conclusion

The Random Password Generator project successfully generates strong and customizable passwords. It demonstrates the use of Python programming concepts such as loops, conditional statements, functions, and built-in modules. The project helps users improve password security and serves as a practical application of Python programming.

References

1. Python Documentation – random Module
2. Python Documentation – string Module
3. Python Official Website
4. Cybersecurity Password Best Practices Resources